ROME Report

LLM-assisted Verilog generation with prompt engineering and verification

Course: LLM4ChipDesign (Spring 2026)

Name: Aviraj Dongare

1. Model Used and Key Generation Settings

LLM Provider / Wrapper

- **Provider:** Anthropic Claude
- **Wrapper selected in notebook:** `Claude` (via `os.environ["MODEL"] = "Claude"`)

Model

- **Model ID:** `claude-3-haiku-20240307`

Key settings / environment

- API key was loaded via **Google Colab Secrets** (not hardcoded in the notebook submission), and assigned to:
  - `os.environ["CLAUDE_API_KEY"] = userdata.get("LLM4ChipDesign")`

Generation settings used in the notebook

- `max_tokens`: 4096 (Claude call inside the notebook wrapper)
- Temperature: default (not explicitly overridden for Claude in this run)
- **Compilation mode:** `iverilog -g2012` (SystemVerilog 2012 mode enabled to ensure consistent compilation behavior)

Verification tools

- **Compiler:** `iverilog`
- **Simulator:** `vvp`
- Each design was verified using **self-checking testbenches** that print a final line containing the substring:
  `passed!`
  (This is required because the notebook's feedback loop checks for the token `passed!` in output.)

2. Hierarchical Decomposition

This project uses the provided ROME hierarchical workflow: generate submodules in order, verify each module with its own testbench, and then generate higher-level modules that instantiate previously generated submodules.

Part I — Provided Mux Hierarchy (Required)

Module list and interfaces:

1. **mux2to1**
   - Interface:
     `input wire in1, input wire in2, input wire select, output wire out`
2. **mux4to1**
   - Interface:
     `input [1:0] sel, input [3:0] in, output reg out`
3. **mux8to1**
   - Interface:
     `input [2:0] sel, input [7:0] in, output reg out`

Instantiation relationships:

- `mux4to1` is generated after `mux2to1` and is intended to be built hierarchically (i.e., it may instantiate `mux2to1` submodules depending on the LLM output and prompt).
- `mux8to1` is generated after `mux4to1` and is intended to build hierarchically on earlier mux modules.
- The notebook's hierarchical prompt includes previous module definitions to encourage reuse and instantiation.

Verification summary (Part I):

- Each mux module had a corresponding self-checking testbench:
  - `mux2to1tb.v`
  - `mux4to1tb.v`
  - `mux8to1tb.v`
- Each module reached:
  - **"Testbench ran successfully"**
  - and the testbench printed:
    - `mux2to1 passed!`
    - `mux4to1 passed!`
    - `mux8to1 passed!`

Artifacts were generated in folders:

- `mux2to1/`, `mux4to1/`, `mux8to1/`
  Each folder contains the final `.v` RTL and iteration logs `log_iter_*.txt`.

Part II — Extension Design (Required): Option A Ripple-Carry Adder Hierarchy

For the extension design, I implemented **Option A**: a ripple-carry adder hierarchy with four modules:

```
half_adder → full_adder → adder4 → adder8
```

This hierarchy is meaningful and strictly hierarchical: each higher module is built from previously generated submodules.

Module list and interfaces (Part II)

1. **half_adder**
   - Interface:
     `input wire a, input wire b, output wire sum, output wire carry`
   - Function: Produces sum and carry for a + b.
2. **full_adder**
   - Interface:
     `input wire a, input wire b, input wire cin, output wire sum, output wire cout`
   - Function: Produces sum and carry-out for a + b + cin.
3. **adder4**
   - Interface:
     `input wire [3:0] a, input wire [3:0] b, input wire cin, output wire [3:0] sum, output wire cout`
   - Function: 4-bit ripple-carry addition (a + b + cin).
4. **adder8** (Top module)
   - Interface:
     `input wire [7:0] a, input wire [7:0] b, input wire cin, output wire [7:0] sum, output wire cout`
   - Function: 8-bit ripple-carry addition (a + b + cin).

Instantiation relationships (Part II)

- **full_adder** instantiates **two half_adders**:
  - First half_adder computes intermediate sum and carry from (a, b)
  - Second half_adder computes final sum and carry from (intermediate_sum, cin)
  - Final carry-out is OR of the two carry signals.
- **adder4** instantiates **four full_adders** in a ripple chain:
  - FA0 uses input cin
  - FA1 uses carry-out of FA0
  - FA2 uses carry-out of FA1
  - FA3 produces final cout
- **adder8 (top)** instantiates **two adder4 modules**:
  - `adder_lo` computes lower 4 bits using cin

    - `adder_hi` computes upper 4 bits using carry-out from lower half
    - This satisfies the assignment requirement:
      Top module instantiates at least two previously generated submodules (i.e., not monolithic).

Verification summary (Part II)

Each Part II module has its own self-checking testbench:

- `half_addertb.v` (exhaustive 4 combinations)
- `full_addertb.v` (exhaustive 8 combinations)
- `adder4tb.v` (exhaustive over all a,b (16×16) and cin (2))
- `adder8tb.v` (edge cases + 500 randomized tests)

All modules reached:

- **"Testbench ran successfully"**
- and printed:
    - `half_adder passed!`
    - `full_adder passed!`
    - `adder4 passed!`
    - `adder8 passed!`

Artifacts were generated in folders:

- `half_adder/`, `full_adder/`, `adder4/`, `adder8/`
  Each folder contains the final `.v` RTL and iteration logs `log_iter_*.txt`.

3. Final Prompts Used (Top Module + Representative Submodule)

The hierarchical notebook constructs prompts that include:

1. a natural language goal
2. a specific module signature
3. previously generated module definitions to encourage instantiation reuse

Below are the **final prompts** used for:

- **Top module:** `adder8`
- **Representative submodule:** `full_adder`

(A) Final Prompt — Top Module (`adder8`)

```
//We are generating a 8-bit ripple-carry adder (top) hierarchically in
Verilog. We have generated half adder, full adder, 4-bit ripple-carry adder,
defined as follows:
module half_adder(input wire a, input wire b, output wire sum, output wire
carry);
// [previous module code included here]
endmodule
module full_adder(input wire a, input wire b, input wire cin, output wire
sum, output wire cout);
// [previous module code included here]
endmodule
module adder4(input wire [3:0] a, input wire [3:0] b, input wire cin, output
wire [3:0] sum, output wire cout);
// [previous module code included here]
endmodule

//Please include the previous module(s) in your response and use them to
hierarchically generate a 8-bit ripple-carry adder (top) defined as:
module adder8(input wire [7:0] a, input wire [7:0] b, input wire cin, output
wire [7:0] sum, output wire cout)
//Insert code here
endmodule
```

Notes:

- The prompt explicitly requests hierarchical generation using prior modules.
- The final generated `adder8` instantiates **two `adder4` modules** to meet the instantiation rule.

(B) Final Prompt — Representative Submodule (`full_adder`)

```
//We are generating a 8-bit ripple-carry adder (top) hierarchically in
Verilog. Please begin by generating a full adder defined as follows:
module full_adder(input wire a, input wire b, input wire cin, output wire
sum, output wire cout)
//Insert code here
endmodule
```

Notes:

- The `full_adder` is a representative submodule because it demonstrates a clean decomposition (built from two `half_adder` blocks and a carry OR).
- This demonstrates meaningful hierarchical RTL structuring rather than monolithic logic.

4. Part III — Concise Iteration Table (Debugging Loop)

For Part III, I documented the iterative improvement process for **`adder8`** using the automatically generated iteration logs:

- `adder8/log_iter_0.txt`
- `adder8/log_iter_1.txt`
- `adder8/log_iter_2.txt`

Iteration Table (adder8)

Iteration	Issue Observed	Evidence (snippet)	Fix Applied	Outcome
0	**Compile failure** due to invalid Verilog syntax in `adder8`	Generated module used C-style braces `{ ... }` and ended with `}` instead of a valid `endmodule` structure.	Regenerated `adder8` using standard Verilog module body syntax and ensured the module terminates with `endmodule`.	Still failed at elaboration (module not recognized).
1	**Elaboration failure:** testbench could not find module definition	`./adder8tb.v:9: error: Unknown module type: adder8`	Regenerated `adder8` using a cleaner, canonical formatting style to ensure the module is fully extracted and visible to the compiler.	Error persisted; another refinement needed.
2	**Canonical module definition fixed module detection/extraction**	Final iteration compiled and ran	Produced a clean `module adder8(...); ... endmodule` (no braces, no malformed endings).	✅ "Testbench ran successfully" + `adder8 passed!`

Key compiler message from Iteration 1

```
./adder8tb.v:9: error: Unknown module type: adder8
2 error(s) during elaboration.
*** These modules were missing:
        adder8 referenced 1 times.
***
```

Final result:

- The notebook reported **"Testbench ran successfully"**
- The testbench printed **`adder8 passed!`**

5. Confirmation All Testbenches Pass + Final Commands Used

Testbenches that passed (Part I)

- `mux2to1tb.v` → `mux2to1 passed!`
- `mux4to1tb.v` → `mux4to1 passed!`
- `mux8to1tb.v` → `mux8to1 passed!`

Testbenches that passed (Part II)

- `half_addertb.v` → `half_adder passed!`
- `full_addertb.v` → `full_adder passed!`
- `adder4tb.v` → `adder4 passed!`
- `adder8tb.v` → `adder8 passed!`

Final compile/sim commands (per module)

The notebook uses this general pattern:

Compile:

```
iverilog -g2012 -o <module_dir>/<module_name> <module_dir>/<module_name>.v
./<module_name>tb.v
```

Run:

```
vvp <module_dir>/<module_name>
```

Examples (explicit):

- `mux2to1`:
  - `iverilog -g2012 -o mux2to1/mux2to1 mux2to1/mux2to1.v ./mux2to1tb.v`
  - `vvp mux2to1/mux2to1`
- `adder8`:
  - `iverilog -g2012 -o adder8/adder8 adder8/adder8.v ./adder8tb.v`
  - `vvp adder8/adder8`

Conclusion

- The notebook can be run top-to-bottom (excluding API key insertion) to reproduce the full workflow.
- Artifacts for every module are stored in a folder named after the module, including:
  - Final RTL: `<module>.v`
  - Iteration logs: `log_iter_*.txt`
  - Conversation log: `log.txt`

These artifacts provide traceability of the LLM generation process and the compile/simulation-based correction loop.